3.c Functions - This & Builders

This Expressions, Type-Safe Builders & Builder Inference

Contents

1 **This Expressions**
Current receiver, qualified this, implicit labels & disambiguation

2 **Type-Safe Builders**
DSL creation, function types with receiver & @DslMarker scope control

3 **Builder Type Inference**
Compiler type inference for generic builders & practical examples

https://kotlinlang.org/docs/this-expressions.html
https://kotlinlang.org/docs/type-safe-builders.html

This Expressions

To denote the current receiver, `this` expressions are used. The meaning depends on the enclosing scope.

Class Member

In a member of a class, **this** refers to the current object of that class.

Extension Function

In an extension function or function literal with receiver, **this** denotes the receiver parameter.

No Qualifier

If **this** has no qualifiers, it refers to the innermost enclosing scope.

Qualified this

To access `this` from an outer scope, use qualified syntax with a label: `this@label`

- Classes get an **implicit label** matching the class name
- Extension functions get a label matching the function name
- Lambda expressions can use explicit labels
- Lambdas without receiver cannot use `this`
- You can skip `this.` when calling member functions

```
class A {  // implicit label @A
    inner class B {  // implicit label @B
        fun Int.foo() {
            val a = this@A    // A's this
            val b = this@B    // B's this
            val c = this      // foo()'s receiver, an Int
            val c1 = this@foo  // also an Int

            val funLit = lambda@ fun String.() {
                val d = this  // receiver, a String
            }

            val funLit2 = { s: String ->
                val d1 = this  // foo()'s receiver
            }
        }
    }
}
```

Type-Safe Builders

Type-safe builders allow creating Kotlin-based **domain-specific languages (DSLs)** suitable for building complex hierarchical data structures in a semi-declarative way.

</>

HTML / XML Markup

Generate structured markup using Kotlin code instead of template engines

UI

UI Layouts

Define user interfaces declaratively (Jetpack Compose, TornadoFX)

API

Configuration DSLs

Ktor routing, Gradle build scripts, Spring bean definitions

Key concept: Builders use `function types with receiver` to provide a context where you can call methods on an implicit `this` receiver.

How Builders Work

1. Define a builder function

```
fun html(
    init: HTML.() -> Unit
): HTML {
    val html = HTML()
    html.init()
    return html
}
```

The `HTML.() -> Unit` is a **function type with receiver**. Inside the lambda, `this` refers to an HTML instance.

The 3-step pattern:

- Take a lambda with receiver as parameter
- Create the receiver object inside the function
- Call init on it and return the configured object

2. Use the DSL

```
val result = html {
    head {
        title { +"Kotlin DSL" }
    }
    body {
        h1 { +"Hello Builders" }
        p { +"Type-safe markup" }
    }
}
```

Each nested function (head, body, h1, p) follows the same pattern: **create** an instance, **initialize** it via the lambda, and **add** it to the parent.
See next slides.

HTML Builders example

```
fun html(init: HTML.() -> Unit): HTML {
    val html = HTML()
    html.init()
    return html
}

class HTML {
    private var headTag: Head? = null
    private var bodyTag: Body? = null

    fun head(init: Head.() -> Unit) {
        headTag = Head(1).apply(init)
    }

    fun body(init: Body.() -> Unit) {
        bodyTag = Body(1).apply(init)
    }

    override fun toString(): String {
        val content = buildString {
            headTag?.let { append(it) }
            bodyTag?.let { append(it) }
        }
        return "<html>\n$content</html>\n"
    }
}
```

```
class Head(private val indent: Int = 0) {
    private var titleTag: Title? = null

    fun title(init: Title.() -> Unit) {
        titleTag = Title(indent + 2).apply(init)
    }

    override fun toString(): String {
        val indentation = "  ".repeat(indent)
        val content = buildString {
            titleTag?.let { append(it) }
        }
        return "$indentation<head>\n$content$indentation</head>\n"
    }
}

class Body(private val indent: Int = 0) {
    private val blocks = mutableListOf<String>()

    fun h1(init: H1.() -> Unit) {
        blocks += H1(indent + 2).apply(init).toString()
    }

    fun p(init: P.() -> Unit) {
        blocks += P(indent + 2).apply(init).toString()
    }

    override fun toString(): String {
        val indentation = "  ".repeat(indent)
        return "$indentation<body>\n${blocks.joinToString("")}"
                                    + "$indentation</body>\n"
    }
}

class Title(indent: Int = 0) : TextTag("title", indent)
class H1(indent: Int = 0) : TextTag("h1", indent)
class P(indent: Int = 0) : TextTag("p", indent)
```

HTML Builders example (cont.)

```kotlin
open class TextTag(private val tagName: String, private val
indent: Int = 0) {
    protected val children = mutableListOf<String>()

    operator fun String.unaryPlus() {
        children += this
    }

    override fun toString(): String {
        val indentation = "  ".repeat(indent)
        val content = children.joinToString("")
        return "$indentation<$tagName>$content</$tagName>\n"
    }
}
```

```kotlin
val result = html {
    head {
        title { +"Kotlin DSL" }
    }
    body {
        h1 { +"Hello Builders" }
        p { +"Type-safe markup" }
    }
}

println(result)
```

```
Output:
<html>
  <head>
      <title>Kotlin DSL</title>
  </head>
  <body>
      <h1>Hello Builders</h1>
      <p>Type-safe markup</p>
  </body>
</html>
```

Scope Control: @DslMarker

Without scope control, nested DSL blocks can accidentally access members of outer receivers. Using a `@DslMarker` annotation prevents this by restricting implicit receiver access.

The Problem

```
html {
    body {
        body { // Accesses outer html!
            // Compiles but wrong!
        }
    }
}
```

The Solution

```
@DslMarker    // define HtmlTagMarker as a DslMarker
@Target(AnnotationTarget.CLASS)
annotation class HtmlTagMarker

@HtmlTagMarker  // class annotated with DslMarker
abstract class Tag(val name: String)
```

- With a `@DslMarker`, only the **nearest** implicit receiver is accessible. Outer receivers require explicit `this@label`
- Annotate the superclass only - all inherited classes are automatically treated as annotated
- DSL markers affect scope control on **type declarations**, **type usages**, and **type aliases**

Builder Type Inference

Builder inference helps the compiler infer type arguments of a generic builder call based on the type information about other calls inside its lambda argument. Enabled by default since **Kotlin 1.7.0**.

Example: buildMap()

```kotlin
val baseMap = mapOf("a" to 1, "b" to 2)

val mapWithTotal = buildMap {
    putAll(baseMap)
    put("Total", baseMap.values.sum())
}
// Inferred: Map<String, Int>

println(mapWithTotal)
// Output: {a=1, b=2, Total=3}
```

```kotlin
public inline fun <K, V> buildMap(
    @BuilderInference builderAction: MutableMap<K, V>.() -> Unit
): Map<K, V>
```

Example: buildList()

```kotlin
val baseList = listOf(1, 2, null, 4, null)

val convertedList = buildList {
    baseList.forEach { value ->
        add(if (value != null) 1 else 0)
    }
}

println(convertedList) // Output: [1, 1, 0, 1, 0]
```

```kotlin
public inline fun <E> buildList(
    @BuilderInference builderAction: MutableList<E>.() -> Unit
): List<E>
```

Requirements for custom builders:

- Builder lambda parameter must be a **function type with receiver**
- Receiver type must use the type arguments the inference should resolve
- Receiver must expose public members/extensions containing those type parameters

Practice Exercises

Practice Exercises

1 Qualified *this*

Solved

Create an outer class with an inner class. Both have a property name. Write an extension function inside the inner class that prints both names using qualified this.

2 Simple DSL Builder

Solved

Implement a menu { } builder DSL that creates a Menu object with items. Use a function type with receiver to allow adding items inside the lambda.

3 @DslMarker Scope Control

Unsolved

Take the menu DSL from exercise 2 and add @DslMarker. Verify that nested builders cannot accidentally access outer receiver members.

4 Builder Inference

Unsolved

Write a generic buildStack { } function where the type parameter is inferred from push() calls inside the lambda. Test with Int and String types.

Solved Exercise 1: Qualified this

Create an outer class with an inner class. Both have a property name. Write an extension function inside the inner class that prints both names using qualified this.

```
class Company(val name: String) {
    inner class Department(val name: String) {

        fun String.printBothNames() {
            println("Company:  ${this@Company.name}")
            println("Dept:     ${this@Department.name}")
            println("Receiver: ${this@printBothNames}")
        }
    }
}

fun main() {
    val dept = Company("JetBrains").Department("Kotlin")
    with(dept) {
        "ext-receiver".printBothNames()
    }
}
// Output:
// Company: JetBrains
// Dept:     Kotlin
// Receiver: ext-receiver
```

Solved Exercise 2: Simple DSL Builder

Implement a menu { } builder DSL that creates a Menu object with items. Use a function type with receiver to allow adding items inside the lambda.

```
data class MenuItem(val name: String, val price: Double)
data class Menu(val items: List<MenuItem>)

// Builder class (receiver)
class MenuBuilder {
    private val items = mutableListOf<MenuItem>()

    fun item(name: String, price: Double) {
        items += MenuItem(name, price)
    }

    fun build() = Menu(items.toList())
}

// Builder function with receiver
fun menu(init: MenuBuilder.() -> Unit): Menu {
    val builder = MenuBuilder()
    builder.init()
    return builder.build()
}
```

```
// Usage
fun main() {
    val lunchMenu = menu {
        item("Salad", 8.50)
        item("Pasta", 12.00)
        item("Coffee", 3.00)
    }
    println(lunchMenu)
}

// Output (real output in the same line…):
// Menu(items=[
//   MenuItem(name=Salad, price=8.5),
//   MenuItem(name=Pasta, price=12.0),
//   MenuItem(name=Coffee, price=3.0)])

// Key: init is MenuBuilder.() -> Unit
// Inside the lambda, `this` is MenuBuilder
// so item() can be called directly.
```

Challange: Build so menus can contain sub-menus